Session 02 - The Caesar Cipher as Functions - Tests

Python Contoso Vault - 8 November 2026

Part 1 - Core: caesar.py

Goal

Turn the Session 01 cipher into functions. Make `encrypt` and `decrypt` work with any whole-number key. Make `brute_force` decrypt a ciphertext with every key from 0 to 25 and return all 26 plaintexts. When the file runs, read a ciphertext and print those 26 plaintexts, one per line.

Run `python caesar.py` in the terminal, type a ciphertext and press Enter. To try one function, open the Python shell in the same folder and run `from caesar import brute_force, decrypt, encrypt`.

Provided

Three encrypted Contoso memos. Each memo uses a different key from 1 to 25. Paste each memo into `python caesar.py` as its input.

Memo	Ciphertext
1	`Jvuavzv ayhcls klzr: aol Rhukf nblza ovbzl pz ivvrlk mvy aol zhslz alht vu 12 Uvcltily.`
2	`Vhgmhlh ybgtgvx: max Zteex hyybvx lixgm Kl 48,500 hg ikbgmxk itixk mabl fhgma. Iextlx vaxvd max ubee.`
3	`Nzyezdz SC: esp Pwwl szxpdelj ecta td zapy ez pgpcj pxawzjpp. Dpyo jzfc ylxp ez esp SC eplx mj Qctolj.`

Instructions

1. Define a function `encrypt` that takes a plaintext as a string and a key as an integer.
   1.1. Turn the key into the key from 0 to 25 that gives the same result. For example, 27 becomes 1 and −1 becomes 25.
   1.2. Shift each letter of the plaintext forward by the key, within its own case.
   1.3. Leave every other character as it is.
   1.4. Return the ciphertext as a string.
2. Define a function `decrypt` that takes a ciphertext as a string and a key as an integer.
   2.1. Decrypt the ciphertext by encrypting it with the opposite key.
   2.2. Return the plaintext as a string.
3. Define a function `brute_force` that takes a ciphertext as a string.
   3.1. Decrypt the ciphertext with each key from 0 to 25, as an integer, in turn.
   3.2. Return the 26 plaintexts as one string, one numbered line per key.
4. When the file runs as a program:
   4.1. Read a ciphertext as a string.
   4.2. Print the 26 numbered lines.

Constraints

- Any whole-number key works, including negative and very large keys. Two keys that differ by 26 give the same result.
- Only the 26 letters of the alphabet shift, in uppercase and lowercase. Each keeps its case. Every other character stays unchanged.
- The three functions return their result. None of them prints or reads input.
- `brute_force` returns 26 lines joined by a newline, with no newline at the end. Line `n` is `NN: <plaintext>`, where `NN` is `n` as two digits (`00` to `25`) and `<plaintext>` is the ciphertext decrypted with key `n`.
- Importing the file reads no input and prints nothing.
- Run as a program, the file reads one ciphertext line with no prompt text and prints exactly the 26 lines of `brute_force`.

Design

This is advice on how to build the code well. No test checks it, and your explain-back looks at it.

- Give every variable and function a type hint.

Input validation

Not required. Assume every text uses English letters, digits, spaces and punctuation only. Any key is a whole number. An empty text is valid.

Test plan

Test	Call or run	Expected output	Actual output	Pass
1	`encrypt("Meet at Contoso HQ!", 3)`	`"Phhw dw Frqwrvr KT!"`		
2	`encrypt("Contoso HQ", 29)`	`"Frqwrvr KT"`		
3	`encrypt("Contoso HQ", -1)`	`"Bnmsnrn GP"`		
4	`encrypt("Contoso HQ", 26)`	`"Contoso HQ"`		
5	`encrypt("Contoso HQ", 10**9)`	`"Oazfaea TC"`		
6	`encrypt("mIxEd CaSe", -30)`	`"iEtAz YwOa"`		
7	`encrypt("Rs 2,500 for the Kandy guest house.", 5)`	`"Wx 2,500 ktw ymj Pfsid lzjxy mtzxj."`		
8	`encrypt("", 7)`	`""`		
9	`decrypt("Phhw dw Frqwrvr KT!", 3)`	`"Meet at Contoso HQ!"`		
10	`decrypt("Contoso HQ", -3)`	`"Frqwrvr KT"`		
11	`decrypt("Contoso HQ", 10**9)`	`"Qcbhcgc VE"`		
12	`brute_force("Frqwrvr")`	26 lines. Line 1 is `00: Frqwrvr`, line 4 is `03: Contoso`, line 26 is `25: Gsrxsws`.		
13	`brute_force("")`	26 lines, `00:` to `25:` , each with one space after the colon.		
14	`python caesar.py` with input `Frqwrvr`	Prints the same 26 lines as test 12.		
15	`import caesar`	Reads no input and prints nothing.		

Done criteria

- [ ] Every test plan row gives exactly the expected result.
- [ ] The script runs from the terminal with `python caesar.py`.

Part 2 - Challenge: best_guess.py

Goal

Find the most likely key for a ciphertext. Compare the letters of each plaintext with how often each letter appears in English text.

Keep `caesar.py` and `english.py` in the same folder as `best_guess.py`. Open the Python shell there, run `from best_guess import best_guess`, then call `best_guess` with each ciphertext.

Provided

`english.py` arrives in this folder when you sync your fork. It defines `expected_freq(letter: str) -> float`. It takes one lowercase letter and returns its share of the letters in English text.

The shares count every letter of Jane Austen's *Pride and Prejudice*, from Project Gutenberg. The count is case-insensitive.

Letter	Share	Letter	Share	Letter	Share	Letter	Share
a	0.07771	h	0.06319	o	0.07452	v	0.01065
b	0.01683	i	0.0711	p	0.01544	w	0.02257
c	0.02523	j	0.00163	q	0.00115	x	0.00178
d	0.04111	k	0.00595	r	0.06025	y	0.02348
e	0.12871	l	0.04079	s	0.06196	z	0.00172
f	0.02237	m	0.02727	t	0.08749		
g	0.01878	n	0.07031	u	0.02799		

Instructions

1. Import `decrypt` from `caesar.py` and `expected_freq` from `english.py`.
2. Define a function `best_guess` that takes a ciphertext as a string.
   2.1. Count how many characters of the ciphertext are letters, as an integer. Uppercase and lowercase letters both count. Spaces, digits and punctuation do not.
   2.2. Return 0 if the ciphertext has no letters.
   2.3. Start the best key at 0, as an integer.
   2.4. Start the best score at infinity, as a float.
   2.5. Try each key from 0 to 25, as an integer, in turn:
      2.5.1. Decrypt the ciphertext with the current key.
      2.5.2. Start the key's score at 0, as a float.
      2.5.3. Go through each of the 26 letters of the alphabet, from a to z, in turn:
         2.5.3.1. Count how many times the letter appears in the plaintext, as an integer. The count is case-insensitive, so `A` and `a` both count as the same letter.
         2.5.3.2. Work out the expected count, as a float: the number of letters in the ciphertext times the letter's share from `expected_freq`.
         2.5.3.3. Add (count - expected count) squared, divided by the expected count, to the key's score.
      2.5.4. Make the key the best key, and its score the best score, if its score is lower than the best score.
   2.6. Return the best key as an integer.

Constraints

- The returned key decrypts the ciphertext.
- When two keys tie, the smaller key wins.
- The function returns its result. It neither prints nor reads input.

Design

This is advice on how to build the code well. No test checks it, and your explain-back looks at it.

- Give every variable and function a type hint.

Input validation

Not required. Any ciphertext is valid, including an empty one.

Test plan

Test	Ciphertext	Letters	Expected output	Actual output	Pass
1	`Gsrxsws wlst xieq: xli ria tvmgi pmwx jsv xli Gspsqfs wxsvi wxevxw sr Qsrhec. Izivc mxiq yrhiv jmzi lyrhvih vytiiw qsziw xs xli jvsrx wlipj.`	113	`4`		
2	`Kyv Tfekfjf kirmvc uvjb sffbj kirzej, sljvj reu xlvjk yfljvj wfi jkrww. Jveu kyv urkvj fw pfli kizg reu kyv kfne pfl nzcc mzjzk kf kyv uvjb.`	111	`17`		
3	`Zlkqlpl pqlzh obmloq: qeb Hxkav txobelrpb eliap qeobb qelrpxka yxdp, kfkb erkaoba ylqqibp lc txqbo xka qtl erkaoba ylubp lc qbx clo qeb pelmp.`	116	`23`		
4	`LXWCXBX YJHAXUU WXCRLN: BJUJARNB OXA WXENVKNA ANJLQ NENAH KJWT JLLXDWC XW CQN CFNWCH OROCQ. LXWCJLC CQN ORWJWLN CNJV FRCQ JWH ZDNBCRXW JKXDC HXDA YJHBURY.`	128	`9`		
5	`Contoso shop team: the new price list for the Colombo store starts on Monday. Every item under five hundred rupees moves to the front shelf.`	113	`0`		
6	empty string	0	`0`		
7	`2026-11-08!`	0	`0`		
8	`Cvaa pgg rvjdl` (true key 1)	12	`15`		

Test 8 shows the method failing on a short ciphertext. It returns 15, but the true key is 1.

Done criteria

- [ ] Every test plan row returns exactly the expected key.